

UNIT I

Introduction to TensorFlow

What is TensorFlow?

- Open-source machine learning framework by **Google**.
- Used to **build, train, and deploy** ML models.
- Supports **machine learning** and **deep learning** algorithms.
- Initially released in **2015** and widely adopted.

Key Features of TensorFlow

- **Flexibility & Scalability:**
 - Works for small models to large neural networks.
 - Supports both **CPU** and **GPU** computations.
- **Eager Execution:**
 - Run operations immediately (introduced in TF 2.x).
 - Easier for **debugging** and experimentation.
- **Comprehensive Ecosystem** (explained next slide).

TensorFlow Ecosystem

- **TensorBoard**: Visualize training and debug performance.
- **TensorFlow Lite**: Run models on **mobile and edge devices**.
- **TensorFlow.js**: Run models in the **browser** using JavaScript.
- **TensorFlow Extended (TFX)**: For **end-to-end ML pipelines**.

Community and Documentation

- Backed by a **large global community**.
- Rich in **tutorials, guides, videos, and pre-trained models**.
- Easy to get help and find examples online.

Applications of TensorFlow – Image Recognition

- Tasks:
 - Object detection
 - Image classification
 - Image segmentation
- Pre-trained models:
 - **Inception**
 - **MobileNet**

Applications of TensorFlow – NLP

- Tasks:
 - Sentiment analysis
 - Language translation
 - Text generation
- Tools & Libraries:
 - **TensorFlow Text**
 - **BERT (Bidirectional Encoder Representations from Transformers)**

Applications – Time Series & Reinforcement Learning

- **Time Series Analysis:**

- Forecasting
- Anomaly detection
- Tools: RNN, LSTM

- **Reinforcement Learning:**

- Agents learn via trial and error
- Use: **TensorFlow Agents**

Ensuring that TensorFlow works

Installation Using pip



Steps to Install with pip

1. Open Terminal/Command Prompt
 - Windows: `Win + R → cmd`
 - Mac/Linux: Open Terminal

Run command:

```
pip install  
tensorflow
```

1. For GPU version (if compatible)

```
pip install  
tensorflow-gpu
```

Installation Using Anaconda

Using Conda (Recommended)

1. Open **Anaconda Prompt** (Windows) or Terminal (macOS/Linux)

Create and activate a virtual environment:

```
conda create --name tf_env python=3.8
```

```
conda activate tf_env
```

2. Install TensorFlow:

```
conda install tensorflow
```

Verifying the Installation

Method 1: Using Python Shell or Notebook

```
import tensorflow as tf  
print(tf.__version__)
```



Expected Output:

Version number like 2.15.0 or similar.

Simple TensorFlow Program

```
import tensorflow as tf

# Create a constant tensor
hello = tf.constant('Hello, TensorFlow!')

# Print the tensor
print(hello)
```

● If successful, you'll see output like:

```
tf.Tensor('Hello, TensorFlow!', shape=(), dtype=string)
```

Running the Program

Ways to Run:

Save code as `test_tf.py` and run:

```
python test_tf.py
```

- OR run directly in **Jupyter Notebook** or **Google Colab**

Using TensorFlow in Google Colab

Steps:

1. Go to <https://colab.research.google.com>
2. Create a new notebook

Run:

```
import tensorflow as tf  
print(tf.__version__)
```

3.  Output should display the version, confirming it works.

Creating Tensors in TensorFlow

What are Tensors?

- Tensors are multi-dimensional arrays used to represent data in TensorFlow.
- You can create tensors in several ways depending on your task.

Constant Tensors

```
import tensorflow as tf
```

```
scalar = tf.constant(3)                # 0D Tensor
```

```
vector = tf.constant([1, 2, 3])        # 1D Tensor
```

```
matrix = tf.constant([[1, 2], [3, 4]]) # 2D Tensor
```

✓ **Constant tensors** are fixed — their values don't change.

Variable Tensors

```
variable = tf.Variable([1, 2, 3])
```

✓ **Variable tensors** can be **modified/updated** during training.

Used in models that need to **learn and update weights**.

Random Tensors

```
random_tensor = tf.random.uniform([2, 3],  
minval=0, maxval=10)
```

✓ Creates a **2x3 tensor** with **random values** between 0 and 10.

Used to **initialize weights randomly** in neural networks.

Tensors of Zeros and Ones

```
zeros = tf.zeros([2, 3])
```

```
ones = tf.ones([2, 3])
```

✓ Creates a **2x3 tensor** filled with all 0s or all 1s.

Used as **initial tensors** in many ML tasks.

Tensor Types

Rank	Name	Shape Example	Explanation
0D	Scalar	<code>()</code>	Single number
1D	Vector	<code>(3,)</code>	Line of values
2D	Matrix	<code>(2, 3)</code>	Table
3D	Tensor	<code>(2, 2, 3)</code>	Stack of tables
4D	Tensor	<code>(2, 2, 2, 2)</code>	Batch of stacked tables
5D	Tensor	<code>(2, 2, 2, 2, 1)</code>	Complex batch (e.g., videos w/ audio)

✓ 0D Tensor (Scalar)

◆ Code:

```
python  
  
scalar = tf.constant(1)
```

◆ Shape:

```
python  
  
( ) # No dimensions
```

✓ 1D Tensor (Vector)

◆ Code:

```
python  
  
vector = tf.constant([1, 2, 3])
```

◆ Shape:

```
python  
  
(3,) # A line of 3 values
```

✓ 2D Tensor (Matrix)

◆ Code:

python

```
matrix = tf.constant([  
    [1, 2, 3],  
    [4, 5, 6]  
])
```

◆ Shape:

python

```
(2, 3) # 2 rows x 3 columns
```

✓ 3D Tensor (Stack of Matrices)

◆ Code:

python

```
tensor_3d = tf.constant([  
    [ # Matrix 1  
        [1, 2, 3],  
        [4, 5, 6]  
    ],  
    [ # Matrix 2  
        [7, 8, 9],  
        [10, 11, 12]  
    ]  
)
```

◆ Shape:

python

```
(2, 2, 3) # 2 matrices of size 2x3
```




4D Tensor (Batch of 3D Tensors)

◆ Code:

python

```
tensor_4d = tf.constant([
    [ # Batch 1
        [
            [1, 2, 3],
            [4, 5, 6]
        ],
        [
            [7, 8, 9],
            [10, 11, 12]
        ]
    ],
    [ # Batch 2
        [
            [13, 14, 15],
            [16, 17, 18]
        ],
        [
            [19, 20, 21],
            [22, 23, 24]
        ]
    ]
])
```

◆ Shape:

python

(2, 2, 2, 3)



✓ 5D Tensor (Sequence of Batches of 3D Tensors)

◆ Code:

python

```
tensor_5d = tf.constant([
    [ # Sequence 1
        [ # Batch 1
            [
                [1, 2, 3],
                [4, 5, 6]
            ],
            [
                [7, 8, 9],
                [10, 11, 12]
            ]
        ],
        [ # Batch 2
            [
                [13, 14, 15],
                [16, 17, 18]
            ],
            [
                [19, 20, 21],
                [22, 23, 24]
            ]
        ]
    ],
    [ # Sequence 2
        [ # Batch 1
            [
                [25, 26, 27],
                [28, 29, 30]
            ],
            [
                [31, 32, 33],
                [34, 35, 36]
            ]
        ],
        [ # Batch 2
            [
                [37, 38, 39],
                [40, 41, 42]
            ],
            [
                [43, 44, 45],
                [46, 47, 48]
            ]
        ]
    ]
])
```



[# Sequence 2

[# Batch 1

[

[25, 26, 27],

[28, 29, 30]

],

[

[31, 32, 33],

[34, 35, 36]

]

],

[# Batch 2

[

[37, 38, 39],

[40, 41, 42]

],

[

[43, 44, 45],

[46, 47, 48]

]

]

]

])

◆ Shape:

python

(2, 2, 2, 2, 3)

What are Operators?

Operators are **functions** in TensorFlow used to perform **mathematical operations** on **tensors**.

1. Addition

python

```
a = tf.constant([1, 2, 3])  
b = tf.constant([4, 5, 6])  
add = tf.add(a, b)  
print(add) # [5 7 9]
```

2. Subtraction

python

```
sub = tf.subtract(a, b)  
print(sub) # [-3 -3 -3]
```

3. Multiplication

python

```
mul = tf.multiply(a, b)  
print(mul) # [ 4 10 18]
```

4. Division

python

```
div = tf.divide(a, b)  
print(div) # [0.25 0.4 0.5]
```



5. Matrix Multiplication

python

```
matrix1 = tf.constant([[1, 2], [3, 4]])  
matrix2 = tf.constant([[5, 6], [7, 8]])  
matmul = tf.matmul(matrix1, matrix2)  
print(matmul)  
# [[19 22]  
# [43 50]]
```

Creating Operators in TensorFlow (Optional: `tf.math`) Alternative Way Using `tf.math`

TensorFlow also provides math functions under the `tf.math` module.

You can use these instead of `tf.add`, `tf.subtract`, etc.



Examples:

```
python
```

```
a = tf.constant([1, 2, 3])
b = tf.constant([4, 5, 6])

# Using tf.math
add = tf.math.add(a, b)
sub = tf.math.subtract(a, b)
mul = tf.math.multiply(a, b)
div = tf.math.divide(a, b)

print(add)  # [5 7 9]
```

Executing Operators with Sessions

```
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()
# Define constants
a = tf.constant(5)
b = tf.constant(3)

# Define an addition operation
add_op = tf.add(a, b)
with tf.Session() as sess:
    # Run the addition operation
    result = sess.run(add_op)
    print(result) # Output: 8
```

Why Sessions Were Used:

1. Explicit Graph Execution:

In TensorFlow 1.x, you first build the graph (just like a blueprint). The session is needed to run it and get results.

2. Resource Management:

Sessions help manage memory and devices (like CPU or GPU) efficiently during execution.

3. Control Over Execution:

Sessions give you control on when and what part of the graph to run

TensorFlow 1.x Example (with Session)

python

```
# Import TensorFlow 1.x
import tensorflow.compat.v1 as tf
tf.disable_v2_behavior()

# Define constants and operations
a = tf.constant(5)
b = tf.constant(3)
add_op = tf.add(a, b)

# Create a session
with tf.Session() as sess:
    # Run the operation within the session
    result = sess.run(add_op)
    print(f"Result with session: {result}")
```

TensorFlow 2.x Example (with Eager Execution)

python

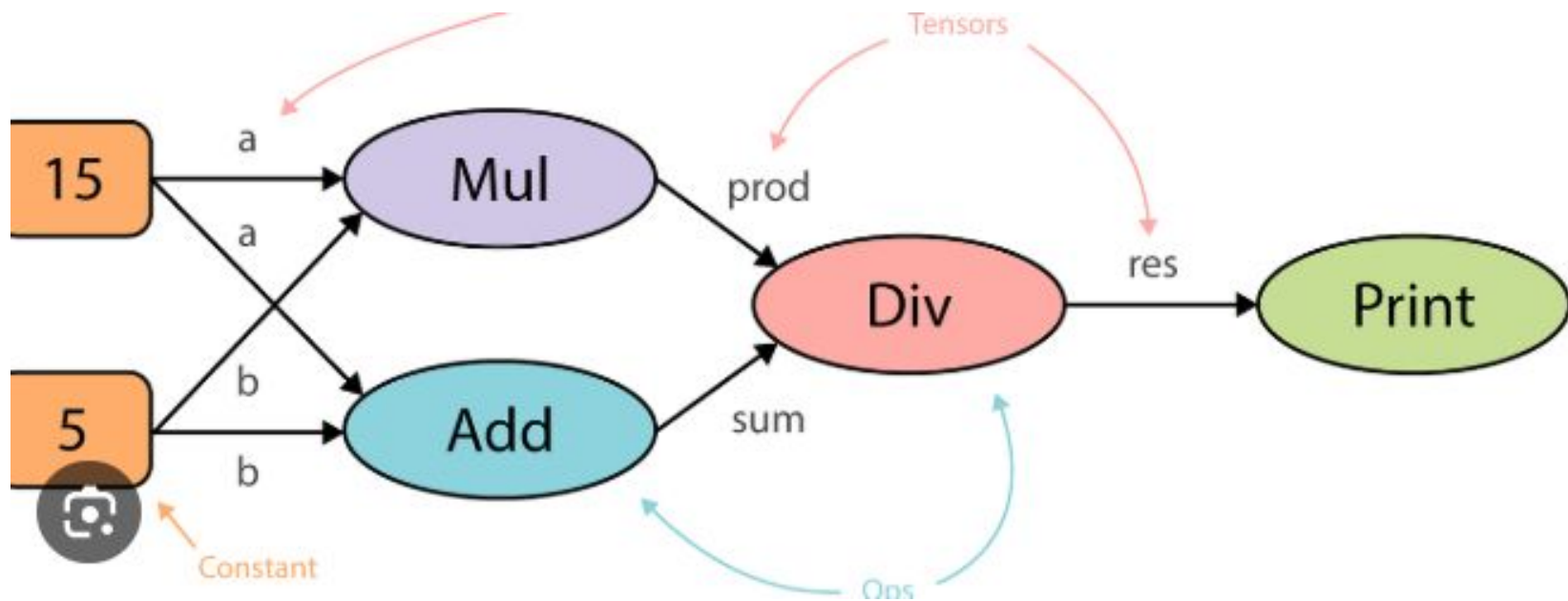
```
# Import TensorFlow 2.x (eager execution is enabled by default)
import tensorflow as tf

# Define constants and operations
a = tf.constant(5)
b = tf.constant(3)
add_result = tf.add(a, b)

# Print the result directly (no session needed)
print(f"Result with eager execution: {add_result.numpy()}")
```

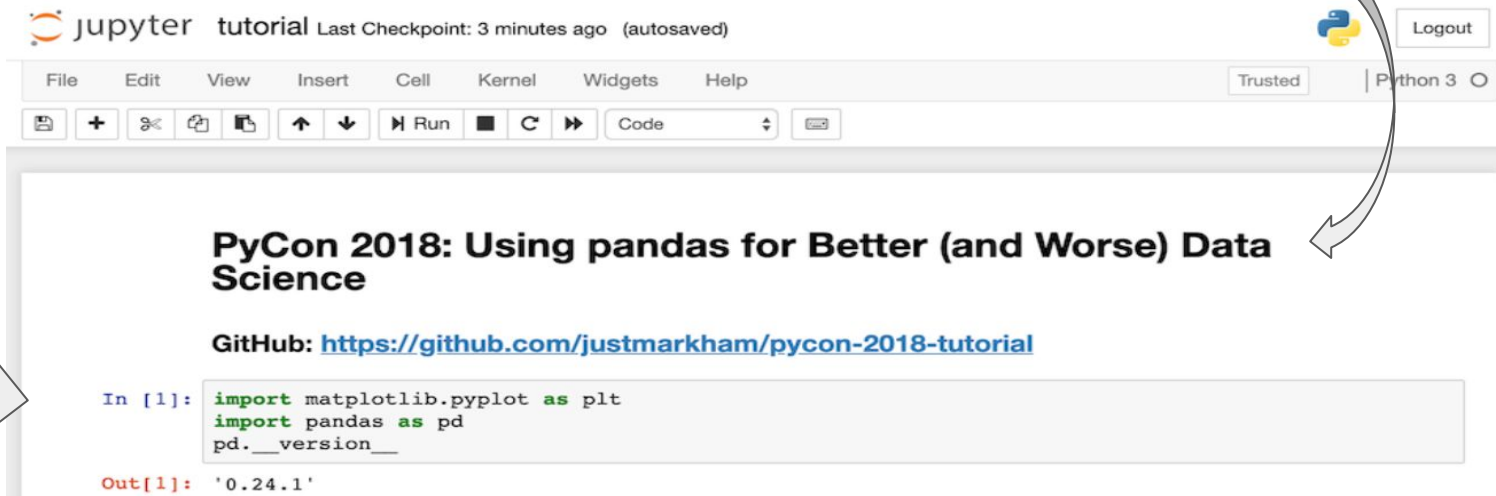
TensorFlow Architecture – Computation Graphs

- Operations are arranged in a **graph**.
- **Nodes** = mathematical operations
- **Edges** = tensors (data) flowing between operations
- Enables **optimization** and **parallelism**



Writing Code in Jupyter,

- **Jupyter Notebook:** An interactive environment that allows you to write and execute Python code in a cell-based format.
- **Code Cells:** Cells where you can write and execute TensorFlow code.
- **Markdown Cells:** Cells used for documentation and explanations.



Using Variables

- **Variables:** Used to store and update values during training or computation.
- **Creating Variables:** Use `tf.Variable()` to create a variable.
- **Updating Variables:** Variables can be updated using assignment operations.

```
# Define a variable  
variable = tf.Variable(10)
```

```
# Update the variable  
variable.assign(15)  
print(variable.numpy())  
# Output: 15
```

Saving Variables

Saving: `tf.train.Checkpoint()` for saving variables.

Example:

```
# Define a variable
```

```
variable = tf.Variable(10)
```

```
# Save the variable
```

```
checkpoint =
```

```
tf.train.Checkpoint(var=variable)
```

```
checkpoint.save('/path/to/save/model')
```

Loading Variables

- **Loading:** Use `tf.train.Checkpoint.restore()` to load saved variables.

Example:

```
# Restore the variable
```

```
checkpoint =
```

```
tf.train.Checkpoint(var=variable)
```

```
checkpoint.restore('/path/to/save/m  
odel-ckpt')
```

```
print(variable.numpy())
```

```
# Output: Restored value
```

Visualizing Data Using TensorBoard in Google Colab:

1. Setup Logging

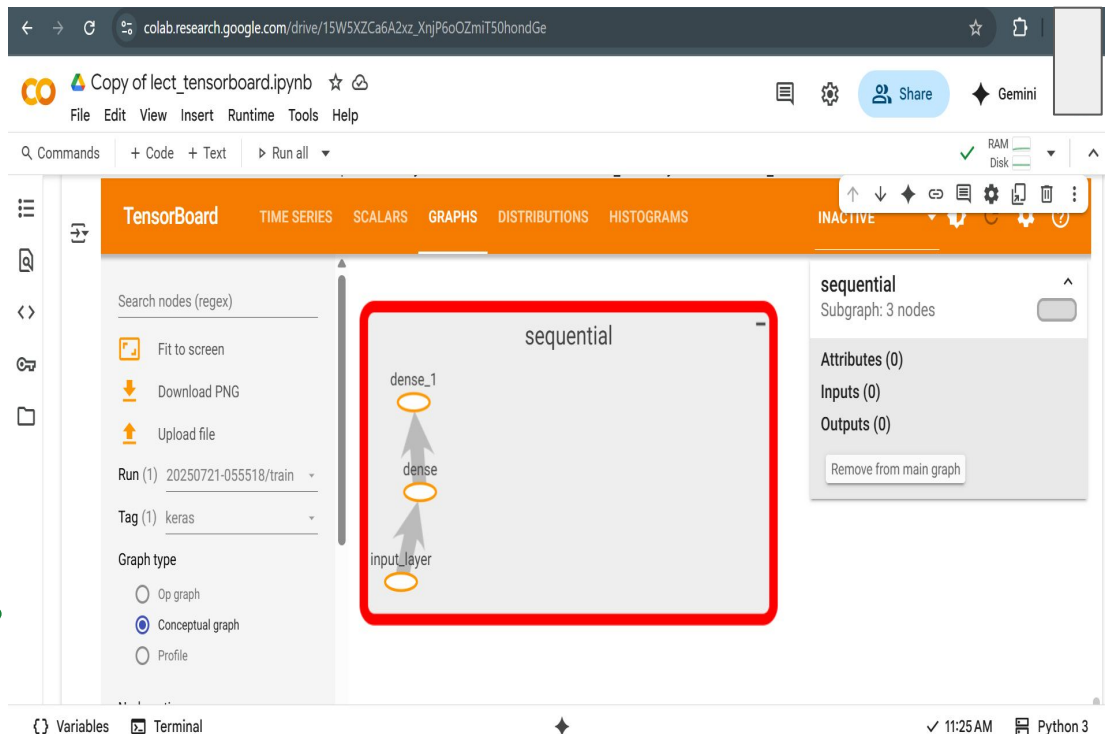
2.Run Your Code

3.Start TensorBoard

In Colab, use:

```
%load_ext tensorboard
```

```
%tensorboard --logdir logs
```



Overview of Keras

- **Keras:** A high-level API for building and training deep learning models. It provides a user-friendly interface to TensorFlow.
- **Features:**
 - **Simplified API:** Easy to use for beginners with fewer lines of code.
 - **Pre-built Layers:** Provides a wide range of layers and optimizers.
 - **Model Building:** Supports sequential and functional API for constructing models.

Architecture of Keras

Sequential API:

- **Description:** Models are created by stacking layers in a linear fashion.
- **Usage:** Ideal for simple, layer-by-layer models.

Functional API:

- **Description:** More flexible, allowing for complex architectures such as multi-input or multi-output models.
- **Usage:** Suitable for models with shared layers or complex layer connections.

Model Subclassing

- **Description:** Model Subclassing in Keras allows for creating custom models by inheriting from `tf.keras.Model` and defining your own `call()` method to specify the forward pass of the model.

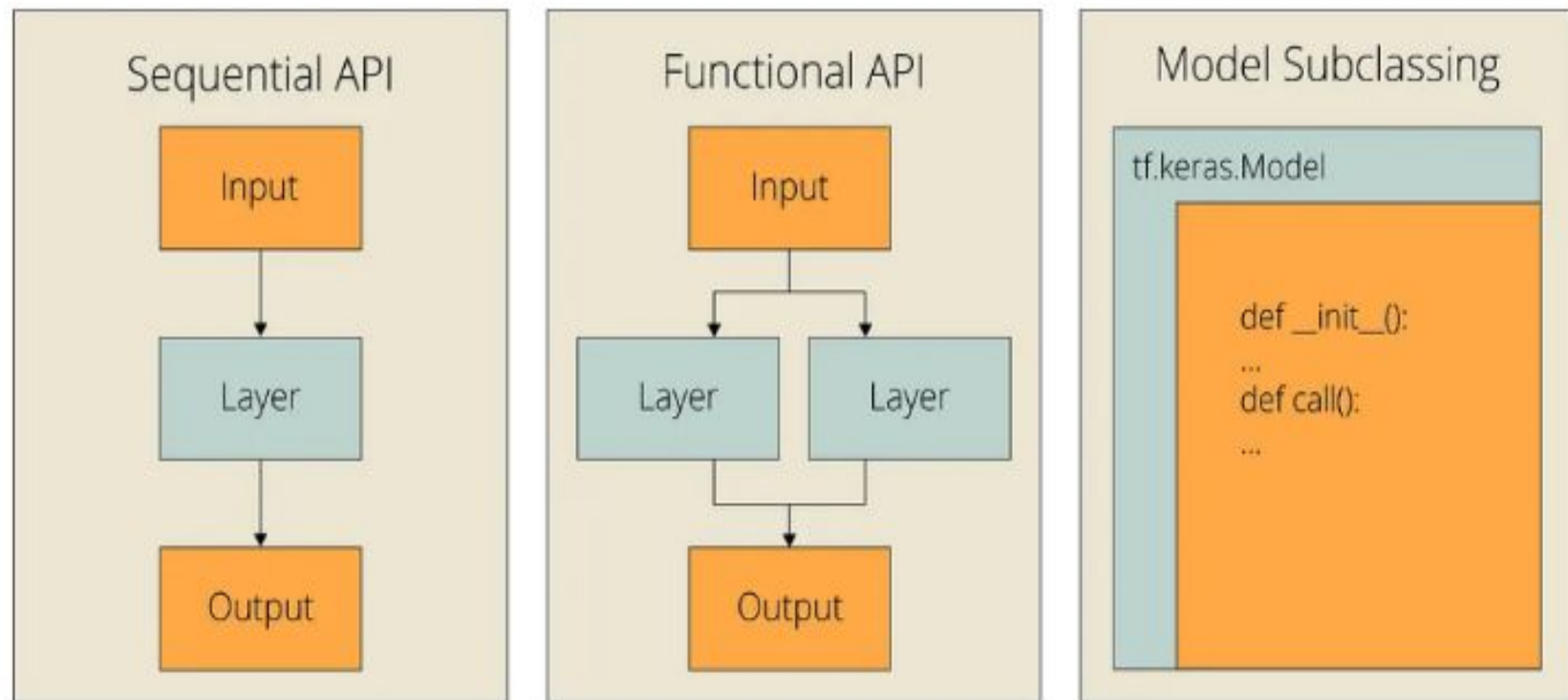


Figure 1. The Sequential API, The Functional API, Model Subclassing Methods Side-by-Side

UNIT II

Introduction to TensorFlow Data API

The TensorFlow Data API helps efficiently **load, preprocess, and transform data** using input pipelines — ideal for large datasets and complex operations.

Key Concepts:

1. **Datasets** – Collections of data (e.g., tensors, files).
2. **Transformations** – Modify data using functions (e.g., map, batch, filter).
3. **Iteration** – Loop through data efficiently.
4. **Combining** – Merge multiple datasets into one.

Creating Datasets

You can create datasets using **tf.data.Dataset** from:

✓ 1. Python List

```
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3, 4, 5])
```

✓ 2. Tensor

```
tensor_data = tf.constant([1, 2, 3, 4, 5])
```

```
dataset = tf.data.Dataset.from_tensor_slices(tensor_data)
```

✓ 3. Text File

```
dataset = tf.data.TextLineDataset("file.txt")
```

✓ 4. TFRecord File

```
dataset = tf.data.TFRecordDataset("file.tfrecords")
```

Transformations in tf.data

Modify and process data using these key operations:

1. Map – Apply a function to each element
2. Filter – Keep elements that meet a condition
3. Batch – Group elements into batches
4. Shuffle – Randomize element order
5. Prefetch – Load data in background for faster training

1. Map

Explanation: Applies a function to each element in the dataset.

Example: Multiply each number by 2.

python

 Copy code

```
import tensorflow as tf

# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Apply the map transformation
mapped_dataset = dataset.map(lambda x: x * 2)

# Print the result
for item in mapped_dataset:
    print(item.numpy())
```

Output:

2
4
6
8
10
12
14
16
18
20

2. Filter

Explanation: Keeps only the elements that meet a certain condition.

Example: Keep numbers greater than 10.

python

 Copy code

```
# Apply the filter transformation
```

```
filtered_dataset = mapped_dataset.filter(lambda x: x > 10)
```

```
# Print the result
```

```
for item in filtered_dataset:  
    print(item.numpy())
```

Output:

12

14

16

18

20

3. Batch

Explanation: Groups elements into smaller sets of a specified size.

Example: Group into batches of size 2.

```
python

# Apply the batch transformation
batched_dataset = filtered_dataset.batch(2)

# Print the result
for batch in batched_dataset:
    print(batch.numpy())
```

Output:

csharp

[12 14]

[16 18]

[20]

4. Shuffle

Explanation: Randomizes the order of elements.

Example: Shuffle the elements of the list.

```
python

# Apply the shuffle transformation
shuffled_dataset = filtered_dataset.shuffle(buffer_size=10)

# Print the result
for item in shuffled_dataset:
    print(item.numpy())
```

Output (The order will vary every time):

16

12

20

18

14

5. Prefetch

Explanation: Optimizes performance by preparing data while the model is training.

Example: Add prefetch to prepare data one step ahead.

```
python

# Apply the prefetch transformation
prefetch_dataset = filtered_dataset.prefetch(buffer_size=tf.data.AUTOTUNE)

# Prefetch is mainly for performance optimization, so results will be the same
for item in prefetch_dataset:
    print(item.numpy())
```

Output:

12

14

16

18

20

Chaining Transformations

Multiple transformations can be chained to create powerful data pipelines.

Program:

```
dataset = tf.data.Dataset.from_tensor_slices(data)

transformed_dataset = dataset.map(lambda x: tf.cast(x, tf.float32) /
50.0).batch(2) .shuffle(5)

for batch in transformed_dataset:

    print(batch.numpy())
```

Iteration

Iteration over datasets can be done using Python's iteration constructs.

Example:

```
# Iterate over the dataset  
for element in transformed_dataset:  
    print(element.numpy())
```

Preprocessing the Data in TensorFlow

Preprocessing involves various steps to transform raw data into a suitable format for model training. Common preprocessing tasks include normalization, standardization, data augmentation, and tokenization.

The goal of normalization:

To scale values into the **range [0, 1]**, using the formula:

$$\text{Normalized value} = \frac{\text{Original value}}{\text{Maximum value}}$$

✓ Example:

For the list `[5, 10, 15, 20, 25]`, the max is **25**.

So:

- $5 \rightarrow 5 / 25 = 0.2$
- $10 \rightarrow 10 / 25 = 0.4$
- $15 \rightarrow 15 / 25 = 0.6$
- $20 \rightarrow 20 / 25 = 0.8$
- $25 \rightarrow 25 / 25 = 1.0$

Now the values are scaled between **0 and 1** — this is **normalization**.

Example:

```
import tensorflow as tf
# Sample data
data = [1, 2, 3, 4, 5]
# Create a dataset from the list
dataset = tf.data.Dataset.from_tensor_slices(data)
# Normalize the data: scaling to [0, 1]
normalized_dataset = dataset.map(lambda x: tf.cast(x, tf.float32) / 5.0)

# Print normalized dataset
print("Normalized dataset:")
for element in normalized_dataset:
    print(element.numpy())
```

Standardization

Standardization transforms data to have a mean of 0 and a standard deviation of 1.

Standardization

Standardization transforms data so that:

- Mean = 0
- Standard Deviation = 1

 Formula:

$$\text{Standardized value} = \frac{(x - \mu)}{\sigma}$$

Where:

- x = original value
- μ = mean of the data
- σ = standard deviation



Math Example:

Let's say the data is:

[10, 20, 30, 40, 50]

1. Mean (μ):

$$\frac{10 + 20 + 30 + 40 + 50}{5} = 30$$

2. Standard Deviation (σ):

$$\sqrt{\frac{(10 - 30)^2 + (20 - 30)^2 + \dots}{5}} = \sqrt{200} \approx 14.14$$

3. Standardized values:

$$10 \rightarrow \frac{(10 - 30)}{14.14} \approx -1.41$$

$$30 \rightarrow \frac{(30 - 30)}{14.14} = 0$$

$$50 \rightarrow \frac{(50 - 30)}{14.14} \approx +1.41$$

```
# Sample data
```

```
data = [1, 2, 3, 4, 5]
```

```
# Calculate mean and standard  
deviation
```

```
mean = tf.reduce_mean(data)
```

```
std_dev = tf.math.reduce_std(data)
```

```
# Create a dataset from the list
```

```
dataset =
```

```
tf.data.Dataset.from_tensor_slices(data)
```

```
# Standardize the data
```

```
standardized_dataset =  
dataset.map(  
    lambda x: (x - mean) / std_dev)
```

```
# Print standardized dataset
```

```
print("Standardized dataset:")
```

```
for element in
```

```
standardized_dataset:
```

```
    print(element.numpy())
```


Data Augmentation (Images)



What is it?

Creating **more training images** by applying random changes to existing ones.



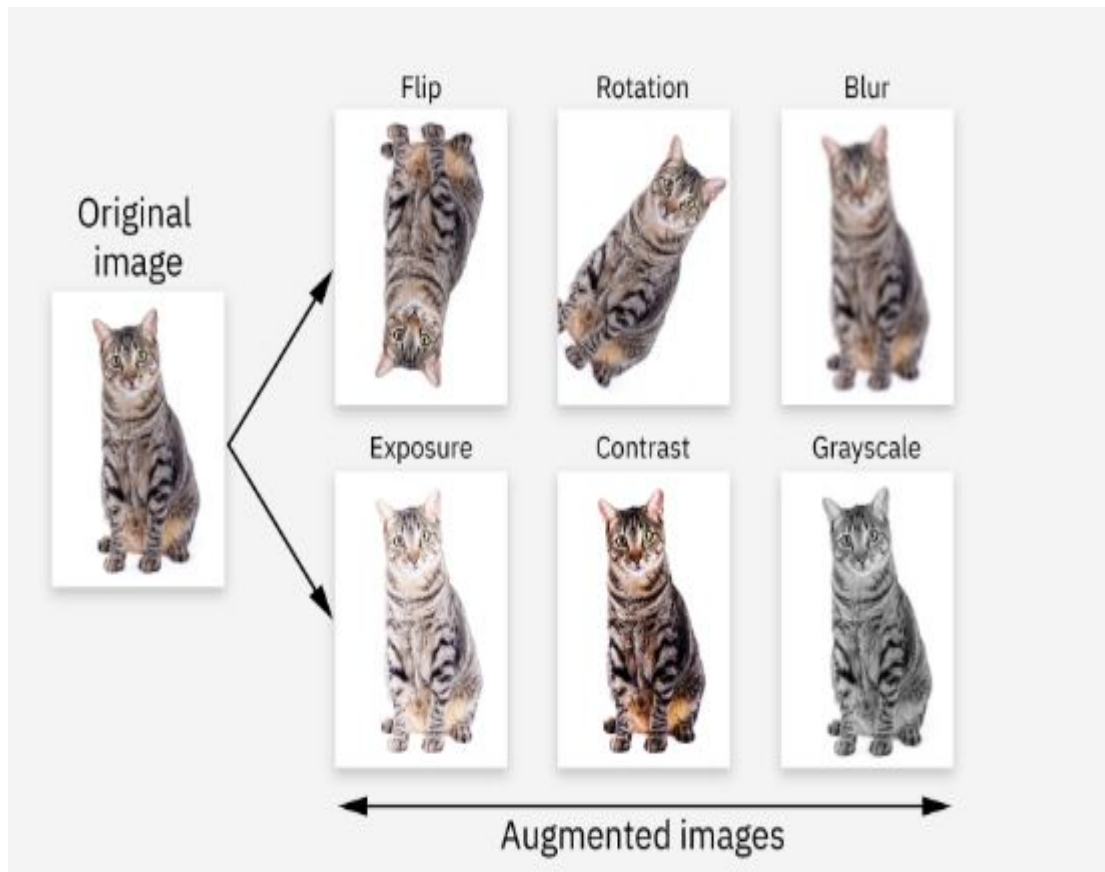
Common Transformations:

- Rotate
- Flip
- Zoom
- Shift
- Brightness change



Why use it?

To help the model learn better and **generalize** by seeing **varied versions** of the same image.



```
import tensorflow as tf

# Sample data: assume images are 28x28 grayscale images
data = tf.random.uniform([5, 28, 28, 1], minval=0, maxval=255,
dtype=tf.float32)

# Create a dataset from the images
dataset = tf.data.Dataset.from_tensor_slices(data)

# Apply data augmentation: random flip
augmented_dataset = dataset.map(lambda x: tf.image.random_flip_left_right(x))

# Print augmented dataset shapes
print("Augmented dataset shapes:")
for element in augmented_dataset:
    print(element.shape)
```

Tokenization (for text)

Tokenization is the process of converting text into sequences of tokens (words or subwords).

```
import tensorflow as tf
```

```
# Sample data
```

```
data = ["Hello TensorFlow", "Data preprocessing is important"]
```

```
# Create a tokenizer
```

```
tokenizer = tf.keras.preprocessing.text.Tokenizer()
```

```
# Fit the tokenizer on the data
```

```
tokenizer.fit_on_texts(data)
```

```
# Convert text to sequences of tokens
```

```
tokenized_data = tokenizer.texts_to_sequences(data)
```

```
# Print tokenized data
```

```
print("Tokenized data:")
```

```
print(tokenized_data)
```